



DEV

Create account



Harry Wynn

Posted on 9 Mar

Why Most Applicant Tracking Systems Are Overkill for Small Teams (So I Built My Own)

#opensource #webdev #node #startup

After 25 years building software and working with recruiting systems, I've developed a strong opinion about ATS platforms:

Most of them are **wildly overbuilt for what small teams actually need.**

Startups and small agencies usually just want to:

- track candidates
- move them through a pipeline
- search resumes
- leave notes

Instead they get massive enterprise systems full of CRM layers, automation engines, and reporting dashboards nobody uses.

So I decided to build something different:

**DEV**[Create account](#)

I call it [Hire Gnome](#).

The Problem: Modern ATS Platforms Are Built for Enterprises

If you've worked with systems like Bullhorn, Greenhouse, Lever, or Workday, you know the pattern.

They are designed around:

- large recruiting teams
- complex workflows
- heavy integrations
- enterprise reporting requirements
- expensive licensing models

All of that complexity trickles down to smaller organizations that don't actually need it.

Typical small-team requirements are much simpler:

- Track candidates
- Store resumes
- Move people through a hiring pipeline
- Add notes
- Search candidates
- Maybe collaborate with one or two other people

That's it.

But instead they end up paying for systems that include:

- complex permission models
- giant CRM layers



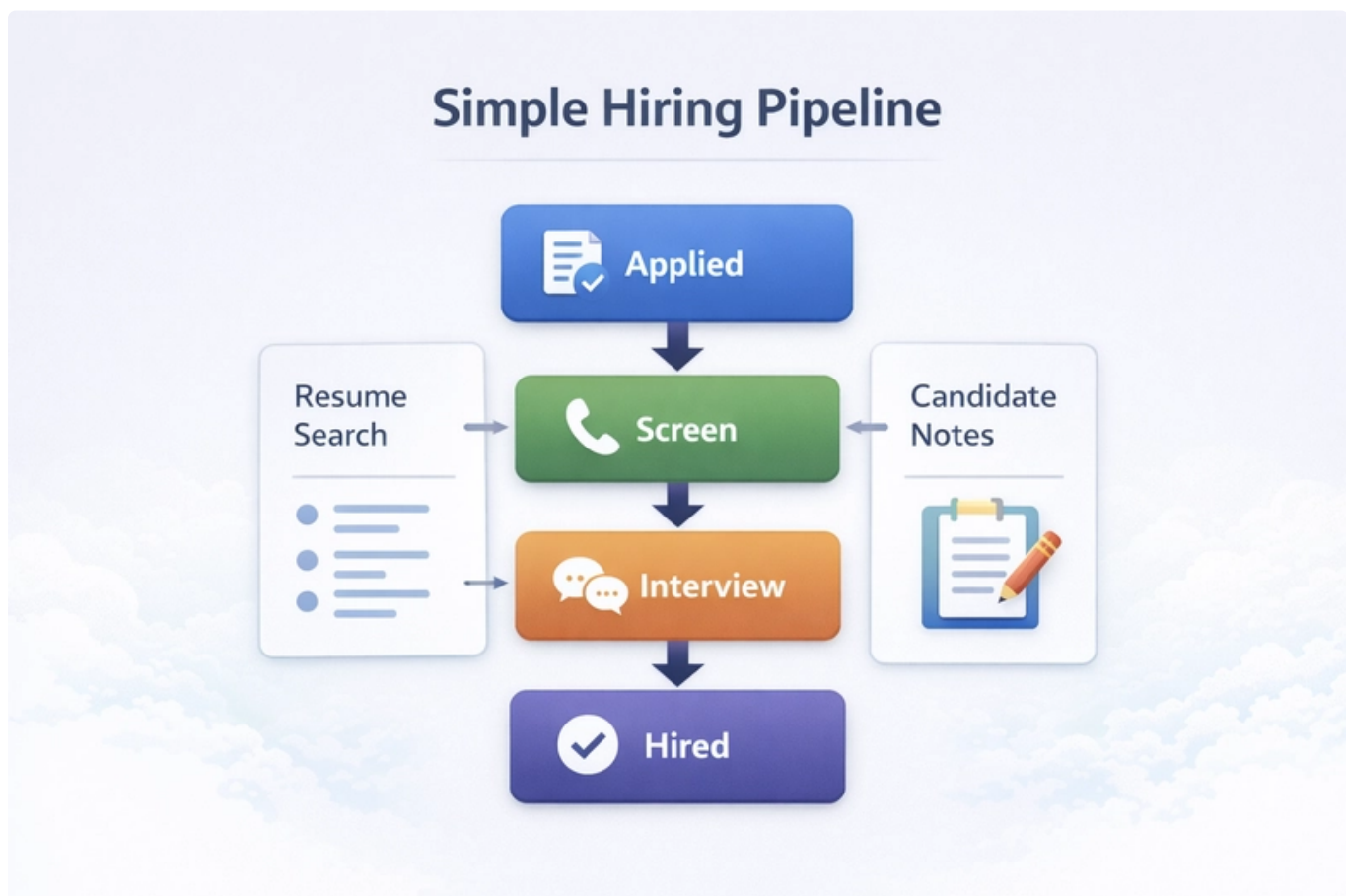
DEV

[Create account](#)

- enterprise compliance tooling
- reporting dashboards nobody actually uses

The result is **tools that are expensive, slow, and hard to customize.**

The Idea: A Simple ATS That Developers Can Actually Run



The core idea behind Hire Gnome is simple:

An ATS that developers can run themselves.

Instead of SaaS lock-in, the project is designed around:

- self-hosting
- open source transparency



DEV

Create account

• easy customization

The mental model is closer to tools like:

- Plausible analytics
- Metabase
- Ghost
- Outline

Install it, run it, own your data.

Small teams get a lightweight ATS.

Developers get something they can extend.

Core Design Principles

Before writing code, I wrote down a few rules the project had to follow.

These guardrails have helped keep the scope from exploding.

1. Simple First

If a feature doesn't help a small team hire someone faster, it probably doesn't belong.

That eliminates a surprising amount of ATS complexity.

No giant CRM layer.

No unnecessary automation engine.

No 50-step workflow configuration.

Just a pipeline and candidates moving through it.

**DEV**[Create account](#)

One of the biggest frustrations with ATS systems is **bad search**.

Recruiters often rely on keyword matching that fails constantly.

Example:

"Frontend engineer with modern JavaScript frameworks"

But the resume says:

"React developer"

Traditional ATS search misses this.

Hire Gnome is being built with **semantic search as a first-class feature**, not an afterthought.

This is an area where modern AI tooling actually makes a meaningful difference.

3. Developers Should Be Able to Extend It

Most ATS systems are black boxes.

If you want to integrate something custom — good luck.

Hire Gnome is intentionally built so developers can easily:

- add integrations
- modify workflows
- extend the data model
- build custom reporting

That means keeping the architecture straightforward.

The Architecture (So Far)



DEV

Create account

That's a good thing.

Backend

A Node.js service responsible for:

- candidate CRUD
- pipeline management
- search indexing
- API access

A clean REST API makes integrations easier down the road.

Database

A relational database holds the core entities:

- Candidates
- Resumes
- Pipeline stages
- Notes
- Activities

This keeps reporting straightforward and avoids unnecessary complexity.

Resume Storage

Resumes are stored as structured text rather than relying purely on raw document formats.

This enables better indexing and search.

A simple transformation pipeline converts uploads into normalized text.



DEV

[Create account](#)

Search Layer

Search is where things get interesting.

Instead of traditional keyword-only search, Hire Gnome uses a hybrid approach:

Keyword + semantic matching

The goal is to allow queries like:

"Find backend engineers with API experience"

and return candidates who mention things like:

- Node
- REST
- GraphQL
- microservices

even if the exact words don't match.

This dramatically improves candidate discovery.

What I'm Trying to Avoid

One of the easiest ways to ruin software is letting it slowly accumulate features.

ATS systems are especially prone to this.

Some examples of things I'm deliberately **not** building (at least not yet):

Sales CRM

Many ATS products try to combine:

- recruiting
- sales pipelines
- account management



DEV

Create account

Hire Gnome focuses on **hiring pipelines only**.

Over-Engineered Workflow Systems

Some ATS platforms allow you to build extremely complex hiring workflows.

In practice, most teams use something like:

Applied
Screen
Interview
Offer
Hired

That's enough.

Flexibility without excessive configuration.

Giant Analytics Dashboards

Analytics are useful, but most teams only care about a few metrics:

- time to hire
- pipeline health
- candidate volume

Those can be added later without building a massive reporting subsystem.

The Open Source Model

Hire Gnome is being built as **open source first**.



DEV

Create account

The core platform is free and self-hosted.

Revenue (eventually) comes from things like:

- managed hosting
- setup and deployment
- custom integrations
- support

Small teams that can run it themselves pay nothing.

Teams that want help can pay for services.

Why the Name "Hire Gnome"

Because every hiring team deserves a small helper behind the scenes doing the hard work.

Also, most good startup names were taken sometime around 2008.

What Comes Next

Right now the focus is on building the core pieces:

- candidate ingestion
- pipeline management
- resume indexing
- semantic search
- simple UI for recruiters

Once those pieces are stable, the roadmap likely includes:

- email ingestion
- calendar integration



DEV

Create account

• API integrations

But the priority is keeping the system **simple and reliable first**.

Lessons So Far

Even early in development, a few lessons have already been clear.

1. Simplicity is harder than complexity

It's surprisingly easy to add features.

It's much harder to say **no** to them.

2. Search is the real value

Tracking candidates is easy.

Finding the right candidate later is the hard part.

3. Open source forces better design

When people can see your code, you naturally build cleaner systems.

The Bottom Line

Recruiting software doesn't have to be bloated enterprise tooling.

For small teams, hiring should be simple:

Collect candidates → evaluate them → hire someone great.

Small.

Transparent.

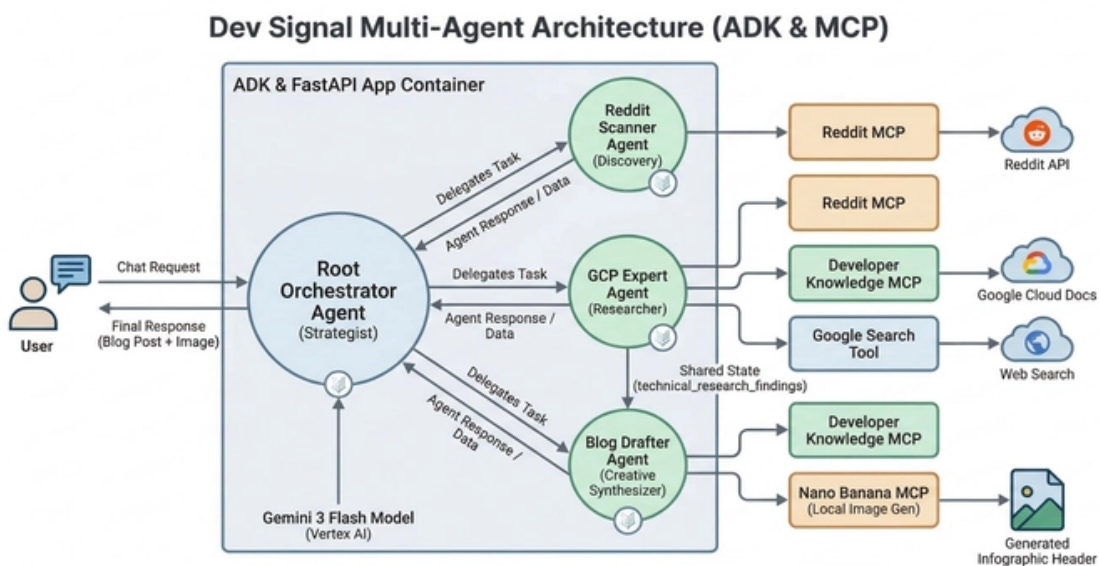
Developer-friendly.

And hopefully useful.

If you're interested in the project or want to follow along as it develops, I'd love to hear feedback from other developers or recruiters who have suffered through the current generation of ATS tools.

DEV The DEV Team PROMOTED

...



Architect A Personalized Multi-Agent System with Long-Term Memory



DEV

Create account

Read More

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)



AWS PROMOTED



Watch Industries LIVE!

Building industry breakthroughs together



Industries
LIVE!





Create account

Building industry breakthroughs together

Discover how the cloud helps businesses adapt, innovate, and grow in real time. Tune in live.

Register Now



Harry Wynn

I'm a senior full-stack developer with 25 years of experience building production web applications and AI-powered systems.

LOCATION

South Carolina

EDUCATION

Penn State

JOINED

13 Nov 2025

More from Harry Wynn

Why don't my database numbers match? I built a tool to find out.

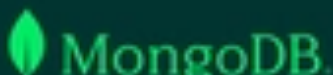
#database #webdev #programming #dataengineering

Building a Production RAG System for Resume Search: What Actually Worked (and What Didn't)

#aws #ai #node #tutorial



MongoDB PROMOTED





DEV

Create account

Join the AI Revolution

Start Building

```
security: 'built-in',
cloud:{
  $in:['AWS', 'Azure', 'GC']
},{
  _id:0,
  try:1
}).forEach((Doc) => {
  printjson(doc);
});{
  'try': 'MongoDB Atlas Today'
}
```

Gen AI apps are built with MongoDB Atlas

MongoDB Atlas is the developer-friendly database for building, scaling, and running gen AI & LLM apps—no separate vector DB needed. Enjoy native vector search, 115+ regions, and flexible document modeling. Build AI faster, all in one place.

Start Free